

GTP5: Programación en C++ para Ciencia e Ingeniería

Estanislao Claucich

Ejercicios

Ejercicio 1	1
Ejercicio 2	2
Ejercicio 3	3
Ejercicio 4	5

[Versión web](#)

A continuación se presentan las soluciones a los ejercicios propuestos en el GTP5.

Ejercicio 1

Consigna

Modificar el ejercicio 1 de la guía anterior (figuras geométricas) agregando una función `label()` genérica que imprima el nombre dinámico del objeto utilizando la herramienta `typeid` de RTTI.

En C++, el polimorfismo permite tratar clases derivadas a través de una referencia o puntero a su clase base. La herramienta `typeid` de la librería `<typeinfo>` permite consultar el tipo de dato subyacente exacto en tiempo de ejecución (*Run-Time Type Identification*).

```
1  #include <typeinfo>
2
3  void label(geoshape_t &g) {
4      cout << "Nombre del objeto (RTTI): " << typeid(g).name() << endl;
5  }
```

Al invocar esta función sobre objetos derivados instanciados, como `square_t` o `circle_t`, el operador evalúa el tipo dinámico e imprime el nombre resultante de la decoración de variables (*name mangling*) generada por el compilador (por ejemplo, el compilador `g++` antepone al nombre original de la clase la cantidad de caracteres que este posee).

```
1  cout << "--- Cuadrado (L=2) ---" << endl;
2  square_t sq(-1.0, -1.0, 1.0, 1.0);
3  label(sq);
4
5  cout << "--- Circulo (R=2) ---" << endl;
6  circle_t circ(0.0, 0.0, 2.0);
7  label(circ);
8
9  cout << "--- Elipse (rx=2, ry=1) ---" << endl;
10 ellipse_t el(0.0, 0.0, 2.0, 1.0);
11 label(el);
```

bash

```
— Cuadrado (L=2) —
Nombre del objeto (RTTI): 8square_t
— Circulo (R=2) —
Nombre del objeto (RTTI): 8circle_t
— Elipse (rx=2, ry=1) —
Nombre del objeto (RTTI): 9ellipse_t
```

Ejercicio 2

Consigna

Implementar la función `void diffsym(const vector<set<int>> &vecset, set<int> &output)` que calcule la diferencia simétrica generalizada de un arreglo de N conjuntos empleando la estructura de datos asociativa `std::map<int, int>`.

La diferencia simétrica generalizada consta en obtener únicamente aquellos elementos que están presentes en exactamente un solo conjunto a lo largo de toda la colección. El uso de `std::map` simplifica la resolución de este problema algorítmico, ya que permite iterar a través de todos los conjuntos e ir contando las ocurrencias independientes de cada valor en un solo ciclo acumulativo:

```

1 void diffsym(const vector<set<int>>& vecset, set<int>& output) {
2     map<int, int> count;
3
4     // Contar apariciones globales
5     for (const auto& s : vecset) {
6         for (int elem : s) {
7             count[elem]++;
8         }
9     }
10
11    // Filtrar diferencia simétrica (frecuencia estricta igual a 1)
12    output.clear();
13    for (const auto& pair : count) {
14        if (pair.second == 1) {
15            output.insert(pair.first);
16        }
17    }
18 }

```

El resultado de las pruebas para tres conjuntos solapados (A={1,2,3,4}, B={3,4,5,6}, C={1,6,7}) demuestra su efectividad:

```
bash
```

```
Conjunto A: { 1, 2, 3, 4 }
```

```
Conjunto B: { 3, 4, 5, 6 }
```

```
Conjunto C: { 1, 6, 7 }
```

```
Diferencia simetrica (elementos que estan en un solo conjunto):
```

```
{ 2 5 7 }
```

Ejercicio 3

i Consigna

Reescribir el ejemplo de los vectores (`vector_t`) de la GTP4 utilizando **templates**, de forma tal que se puedan instanciar vectores dinámicos (`fullvec_t` y `sparse_t`) de cualquier tipo primitivo (incluyendo clases que tengan sobrecargado su operador `+` y `<` como el objeto `std::string`).

El sistema polimórfico de clases abstractas desarrollado en prácticas previas se reescribió en este ejercicio sustituyendo el tipo numérico rígido y estricto (`double`) por un parámetro de plantilla paramétrica `typename T`:

```
1 template <typename T>
2 class vector_t {
3     public:
4         virtual int size() const = 0;
5         virtual T operator[] (int i) const = 0;
6 };
```

Las funciones auxiliares globales que evalúan la jerarquía también fueron transformadas a funciones template (`sum`, `non_null`, `max`, `min` y `reduce`), las cuales logran deducir el tipo subyacente y devuelven este mismo tipo paramétrico original. Para suplir la carencia algorítmica del número nulo absoluto `0.0` en operaciones aritméticas, las nuevas funciones asumen instanciar dinámicamente un constructor vacío `T()` como valor neutro de inicio (ej: `0` para enteros y `""` para strings).

```
1 template <typename T>
2 T sum(const vector_t<T> &v) {
3     if (v.size() == 0) return T();
4     T total = v[0];
5     for (int i = 1; i < v.size(); ++i) {
6         total = total + v[i];
7     }
8     return total;
9 }
```

A través de esta migración estructural, el mismo código fuente de clases abstractas y métodos algorítmicos se recicla completamente, pudiendo utilizarse tanto para operaciones analíticas vectoriales estándar como para realizar la concatenación continua de palabras dentro de un vector de strings:

```
1 // Prueba con double
2 fullvec_t<double> fv_d({1.0, 0.0, -3.0, 4.0, 0.0});
3 cout << "--- Full Vector <double> ---" << endl;
4 cout << "Non nulls: " << non_null(fv_d) << " | Sum: " << sum(fv_d) << " | Max: " << max(fv_d);
5
6 // Prueba con string (operator+ concatena, > y < son alfabéticos)
7 fullvec_t<string> fv_s({"Hola", "", "Mundo", "!", ""});
8 cout << "\n--- Full Vector <string> ---" << endl;
9 cout << "Non nulls: " << non_null(fv_s) << " | Sum: " << sum(fv_s) << " | Max: " << max(fv_s);
```

```

10
11 // Prueba Sparse con int
12 sparse_t<int> spv_i(10, {2, 5, 8}, {100, -50, 25});
13 cout << "\n--- Sparse Vector <int> ---" << endl;
14 cout << "Non nulls: " << non_null(sp_v_i) << " | Sum: " << sum(sp_v_i) << " | Max: " << max(sp_v_i) << endl;
15
16 // Prueba Reduce con string
17 sum_t<string> string_sum_op;
18 cout << "\n--- Prueba Reduce (Full Vector <string>) ---" << endl;
19 cout << "Reduce Sum: " << reduce(fv_s, string_sum_op) << endl;

```

```
bash
```

```

— Full Vector —
Non nulls: 3 | Sum: 2 | Max: 4 | Min: -3
— Full Vector —
Non nulls: 3 | Sum: HolaMundo! | Max: Mundo | Min:
— Sparse Vector —
Non nulls: 3 | Sum: 75 | Max: 100 | Min: -50
— Prueba Reduce (Full Vector ) —
Reduce Sum: HolaMundo!

```

Ejercicio 4

Consigna

A partir del vector genérico de punteros `vector<player_t*> players` de la guía anterior (simulador Tic-Tac-Toe), iterar a todos los agentes creados al azar y filtrar con exclusividad aquellos que correspondan a la clase derivada `dumb_t`. Hacerlo utilizando dos variantes del RTTI: `dynamic_cast` con punteros y `dynamic_cast` con referencias (involucrando el uso de excepciones).

El operador `dynamic_cast` evalúa en tiempo de ejecución si la conversión (*downcasting*) entre un objeto polimórfico padre e hijo en un árbol jerárquico es válida o no.

En la primera implementación iterativa por medio de **punteros**, una falla en el proceso de *casting* da como resultado la asignación inofensiva de una región nula de memoria (`nullptr`), por lo que el uso de un condicional lógico simple resuelve el problema fácilmente:

```

1 cout << "--- Filtrando dumb_t usando dynamic_cast<> con PUNTEROS ---" << endl;
2 for(player_t* p : tournament) {
3     dumb_t* dumb_ptr = dynamic_cast<dumb_t*>(p);
4     if (dumb_ptr != nullptr) {
5         cout << "Encontrado: " << dumb_ptr->label() << endl;
6     }
7 }

```

Por otro lado, cuando se obtiene una *referencia nula*, al tratar de desreferenciarla, el lenguaje se encarga de abortar y alertar sobre la operación errónea lanzando una excepción `std::bad_cast` estandarizada. Para tener en cuenta estos casos, se implementa un bloque de excepción `try-catch`.

```

1 cout << "\n--- Filtrando dumb_t usando dynamic_cast<> con REFERENCIAS y EXCEPCIONES ---" << endl;
2 for(player_t* p : tournament) {
3     try {
4         dumb_t& dumb_ref = dynamic_cast<dumb_t&>(*p);
5         cout << "Encontrado: " << dumb_ref.label() << endl;
6     } catch (const bad_cast& e) {
7         // Ignorado silenciosamente: el objeto no recae en la clase dumb_t
8     }
9 }

```

Probando un ejemplo con varios jugadores de distintos tipos:

```

1 srand(time(NULL));
2 vector<player_t*> tournament;
3
4 // Generamos 8 jugadores de tipos aleatorios
5 for(int i = 1; i <= 8; i++) {
6     int r = rand() % 5;
7     string id = to_string(i);
8     if (r == 0) tournament.push_back(new dumb_t(id));
9     else if (r == 1) tournament.push_back(new random_t(id));
10    else if (r == 2) tournament.push_back(new agresive_t(id));
11    else if (r == 3) tournament.push_back(new minimax_t(id));
12    else tournament.push_back(new rule_based_t(id));
13 }
14
15 // Imprimimos la lista completa solo para contexto
16 cout << "=== Lista completa de jugadores generados ===" << endl;

```

```

17 for(player_t* p : tournament) {
18     cout << p->label() << endl;
19 }
20 cout << "=====\n" << endl;
21
22 // 1er Método: Usando dynamic_cast<> con punteros
23 cout << "--- Filtrando dumb_t usando dynamic_cast<> con PUNTEROS ---" << endl;
24 for(player_t* p : tournament) {
25     dumb_t* dumb_ptr = dynamic_cast<dumb_t*>(p);
26     if (dumb_ptr != nullptr) {
27         cout << "Encontrado: " << dumb_ptr->label() << endl;
28     }
29 }
30
31 cout << "\n--- Filtrando dumb_t usando dynamic_cast<> con REFERENCIAS y EXCEPCIONES ---" << endl;
32 for(player_t* p : tournament) {
33     try {
34         // Se desreferencia p para obtener una referencia
35         dumb_t& dumb_ref = dynamic_cast<dumb_t&>(*p);
36         cout << "Encontrado: " << dumb_ref.label() << endl;
37     } catch (const bad_cast& e) {
38         // El objeto subyacente no es un dumb_t, la excepción es capturada silenciosamente.
39     }
40 }
41
42 // Limpieza de memoria
43 for(player_t* p : tournament) {
44     delete p;
45 }

```

bash

=== Lista completa de jugadores generados ===

```

agresive_1
random_2
agresive_3
dumb_4
agresive_5
dumb_6
minimax_7
rule_based_8
rule_based_9

```

```

random_10
rule_based_11
minimax_12
dumb_13
random_14
dumb_15
dumb_16
=====
— Filtrando dumb_t usando dynamic_cast<> con PUNTEROS —
Encontrado: dumb_4
Encontrado: dumb_6
Encontrado: dumb_13
Encontrado: dumb_15
Encontrado: dumb_16
— Filtrando dumb_t usando dynamic_cast<> con REFERENCIAS y EXCEPCIONES
—
Encontrado: dumb_4
Encontrado: dumb_6
Encontrado: dumb_13
Encontrado: dumb_15
Encontrado: dumb_16

```